YOLO11 Bean Identification & Quality Assessment

This project utilizes the **YOLO11** (You Only Look Once) object detection architecture to automate the process of identifying and classifying beans into two categories: **Good Beans** and **Bad Beans**. It is designed to assist in quality control for agricultural or food processing applications.

📂 Project Structure

```
beans/
├── config/              # Configuration files (data.yaml)
├── data/
│   └── yolo/            # Dataset (Images and YOLO format Labels)
├── runs/                # Training results and inference predictions
├── scripts/             # Python automation scripts
│   ├── inference.py     # Real-time inference (Webcam/Video/Flask)
│   ├── train.py         # Main training script
│   └── run_trained_model.py # Simple inference test script
├── trained_model/       # Folder for final .pt weight files
└── README.md            # You are here
```

🚀 Getting Started

1. Installation

Ensure you have Python 3.10+ installed. Install the necessary dependencies using pip:

```
pip install ultralytics opencv-python flask numpy psutil
```

2. Dataset Setup

The model distinguishes between:

- **Class 0: Good Beans** (Healthy, whole beans)
- **Class 1: Bad Beans** (Damaged, discolored, or shriveled beans)

Labels are stored in standard YOLO `.txt` format. Paths for training and validation are managed in `config/data.yaml`.

🛠️ Usage

Training

To train the model on your local machine, run:

```
python scripts/train.py
```

Note: If training on a CPU-only machine, the script is optimized with `workers=0` and `imgsz=320` to ensure stability.

Real-Time Inference

To run detection using your webcam or a local video file:

```
python scripts/inference.py --model trained_model/my_model.pt --source 0
```

Web API Mode

To start a Flask server that processes base64-encoded images:

```
python scripts/inference.py --model trained_model/my_model.pt --web
```

📊 Performance Analysis

Current testing indicates that the "Bad Bean" detection requires a larger variety of training samples to improve its recall, as the visual features of defects can vary significantly.

Optimization To-Do:

- [ ] Increase "Bad Bean" dataset sample size.
- [ ] Refine confidence thresholds (currently default at 0.25).
- [ ] Implement early stopping to prevent overfitting.

⚖️ Troubleshooting & Linting

If you encounter "no-member" errors for `cv2` in VS Code (Pylint), add the following to your VS Code settings:

```
"pylint.args": ["--generated-members=cv2.*"]
```

Author: YOLO Bean Quality Team **Version:** 1.0.0